

Are Transformers Effective for Time Series Forecasting?

Ailing Zeng^{1*}, Muxi Chen^{1*}, Lei Zhang², Qiang Xu¹

¹The Chinese University of Hong Kong

²International Digital Economy Academy (IDEA)

{alzeng, mxchen21, qxu}@cse.cuhk.edu.hk

{leizhang}@idea.edu.cn

Abstract

Recently, there has been a surge of Transformer-based solutions for the long-term time series forecasting (LTSF) task. Despite the growing performance over the past few years, we question the validity of this line of research in this work. *Specifically, Transformers is arguably the most successful solution to extract the semantic correlations among the elements in a long sequence. However, in time series modeling, we are to extract the temporal relations in an ordered set of continuous points. While employing positional encoding and using tokens to embed sub-series in Transformers facilitate preserving some ordering information, the nature of the permutation-invariant self-attention mechanism inevitably results in temporal information loss.*

To validate our claim, we introduce a set of embarrassingly simple one-layer linear models named LTSF-Linear for comparison. Experimental results on nine real-life datasets show that LTSF-Linear surprisingly outperforms existing sophisticated Transformer-based LTSF models in all cases, and often by a large margin. Moreover, we conduct comprehensive empirical studies to explore the impacts of various design elements of LTSF models on their temporal relation extraction capability. We hope this surprising finding opens up new research directions for the LTSF task. We also advocate revisiting the validity of Transformer-based solutions for other time series analysis tasks (e.g., anomaly detection) in the future. Code is available at: <https://github.com/cure-lab/LTSF-Linear>.

1. Introduction

Time series are ubiquitous in today’s data-driven world. Given historical data, time series forecasting (TSF) is a long-standing task that has a wide range of applications, including but not limited to traffic flow estimation, en-

ergy management, and financial investment. Over the past several decades, TSF solutions have undergone a progression from traditional statistical methods (e.g., ARIMA [1]) and machine learning techniques (e.g., GBRT [11]) to deep learning-based solutions, e.g., Recurrent Neural Networks [15] and Temporal Convolutional Networks [3, 17].

Transformer [26] is arguably the most successful sequence modeling architecture, demonstrating unparalleled performances in various applications, such as natural language processing (NLP) [7], speech recognition [8], and computer vision [19, 29]. Recently, there has also been a surge of Transformer-based solutions for time series analysis, as surveyed in [27]. Most notable models, which focus on the less explored and challenging long-term time series forecasting (LTSF) problem, include LogTrans [16] (NeurIPS 2019), Informer [30] (AAAI 2021 Best paper), Autoformer [28] (NeurIPS 2021), Pyraformer [18] (ICLR 2022 Oral), Triformer [5] (IJCAI 2022) and the recent FEDformer [31] (ICML 2022).

The main working power of Transformers is from its multi-head self-attention mechanism, which has a remarkable capability of extracting semantic correlations among elements in a long sequence (e.g., words in texts or 2D patches in images). However, self-attention is *permutation-invariant* and “anti-order” to some extent. While using various types of positional encoding techniques can preserve some ordering information, it is still inevitable to have temporal information loss after applying self-attention on top of them. This is usually not a serious concern for semantic-rich applications such as NLP, e.g., the semantic meaning of a sentence is largely preserved even if we reorder some words in it. However, when analyzing time series data, there is usually a lack of semantics in the numerical data itself, and we are mainly interested in modeling the temporal changes among *a continuous set of points*. That is, the order itself plays the most crucial role. Consequently, we pose the following intriguing question: ***Are Transformers really effective for long-term time series forecasting?***

Moreover, while existing Transformer-based LTSF so-

*Equal contribution

lutions have demonstrated considerable prediction accuracy improvements over traditional methods, in their experiments, all the compared (non-Transformer) baselines perform autoregressive or iterated multi-step (IMS) forecasting [1, 2, 22, 24], which are known to suffer from significant error accumulation effects for the LTSF problem. Therefore, in this work, we challenge Transformer-based LTSF solutions with direct multi-step (DMS) forecasting strategies to validate their real performance.

Not all time series are predictable, let alone long-term forecasting (e.g., for chaotic systems). We hypothesize that long-term forecasting is only feasible for those time series with a relatively clear trend and periodicity. As linear models can already extract such information, we introduce a set of embarrassingly simple models named *LTSF-Linear* as a new baseline for comparison. *LTSF-Linear* regresses historical time series with a one-layer linear model to forecast future time series directly. We conduct extensive experiments on nine widely-used benchmark datasets that cover various real-life applications: traffic, energy, economics, weather, and disease predictions. Surprisingly, our results show that *LTSF-Linear* outperforms existing complex Transformer-based models *in all cases, and often by a large margin* (20% $\sim$ 50%). Moreover, we find that, in contrast to the claims in existing Transformers, most of them fail to extract temporal relations from long sequences, i.e., the forecasting errors are not reduced (sometimes even increased) with the increase of look-back window sizes. Finally, we conduct various ablation studies on existing Transformer-based TSF solutions to study the impact of various design elements in them.

To sum up, the contributions of this work include:

- To the best of our knowledge, this is the first work to challenge the effectiveness of the booming Transformers for the long-term time series forecasting task.
- To validate our claims, we introduce a set of embarrassingly simple one-layer linear models, named *LTSF-Linear*, and compare them with existing Transformer-based LTSF solutions on nine benchmarks. *LTSF-Linear* can be a new baseline for the LTSF problem.
- We conduct comprehensive empirical studies on various aspects of existing Transformer-based solutions, including the capability of modeling long inputs, the sensitivity to time series order, the impact of positional encoding and sub-series embedding, and efficiency comparisons. Our findings would benefit future research in this area.

With the above, we conclude that *the temporal modeling capabilities of Transformers for time series are exaggerated, at least for the existing LTSF benchmarks*. At the same time, while *LTSF-Linear* achieves a better prediction

accuracy compared to existing works, it merely serves as a simple baseline for future research on the challenging long-term TSF problem. With our findings, we also advocate revisiting the validity of Transformer-based solutions for other time series analysis tasks in the future.

2. Preliminaries: TSF Problem Formulation

For time series containing C variates, given historical data $\mathcal{X} = \{X_1^t, \dots, X_C^t\}_{t=1}^L$, wherein L is the look-back window size and X_i^t is the value of the i_{th} variate at the t_{th} time step. The time series forecasting task is to predict the values $\hat{\mathcal{X}} = \{\hat{X}_1^t, \dots, \hat{X}_C^t\}_{t=L+1}^{L+T}$ at the T future time steps. When $T > 1$, **iterated multi-step (IMS)** forecasting [23] learns a single-step forecaster and iteratively applies it to obtain multi-step predictions. Alternatively, **direct multi-step (DMS)** forecasting [4] directly optimizes the multi-step forecasting objective at once.

Compared to DMS forecasting results, IMS predictions have smaller variance thanks to the autoregressive estimation procedure, but they inevitably suffer from error accumulation effects. Consequently, **IMS forecasting is preferable when there is a highly-accurate single-step forecaster, and T is relatively small**. In contrast, DMS forecasting generates more accurate predictions when it is hard to obtain an unbiased single-step forecasting model, or T is large.

3. Transformer-Based LTSF Solutions

Transformer-based models [26] have achieved unparalleled performances in many long-standing AI tasks in natural language processing and computer vision fields, thanks to the effectiveness of the multi-head self-attention mechanism. This has also triggered lots of research interest in Transformer-based time series modeling techniques [20, 27]. In particular, a large amount of research works are dedicated to the LTSF task (e.g., [16, 18, 28, 30, 31]). Considering the ability to capture long-range dependencies with Transformer models, most of them focus on the less-explored long-term forecasting problem ($T \gg 1$)¹.

When applying the vanilla Transformer model to the LTSF problem, it has some limitations, including the quadratic time/memory complexity with the original self-attention scheme and error accumulation caused by the autoregressive decoder design. Informer [30] addresses these issues and proposes a novel Transformer architecture with reduced complexity and a DMS forecasting strategy. Later, more Transformer variants introduce various time series features into their models for performance or efficiency improvements [18, 28, 31]. We summarize the design elements of existing Transformer-based LTSF solutions as follows (see Figure 1).

¹Due to page limit, we leave the discussion of non-Transformer forecasting solutions in the Appendix.

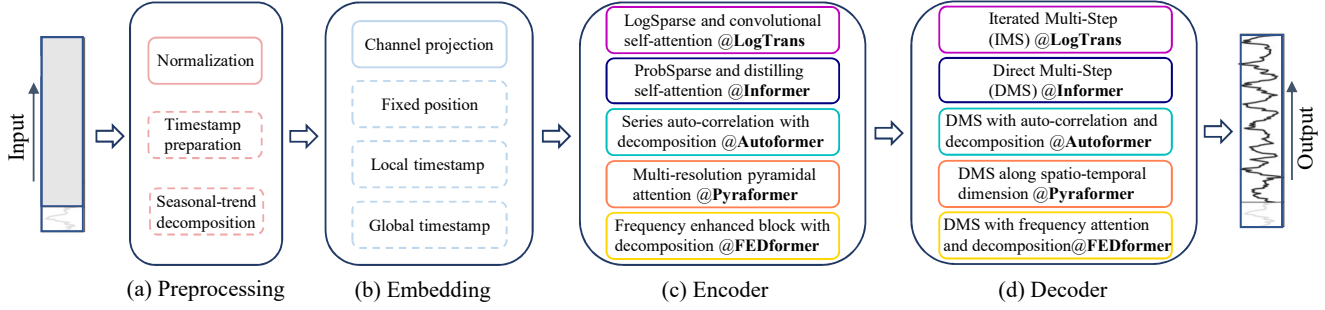


Figure 1. The pipeline of existing Transformer-based TSF solutions. In (a) and (b), the solid boxes are essential operations, and the dotted boxes are applied optionally. (c) and (d) are distinct for different methods [16, 18, 28, 30, 31].

Time series decomposition: For data preprocessing, normalization with zero-mean is common in TSF. Besides, Autoformer [28] first applies seasonal-trend decomposition behind each neural block, which is a standard method in time series analysis to make raw data more predictable [6, 13]. Specifically, they use a moving average kernel on the input sequence to extract the *trend-cyclical* component of the time series. The difference between the original sequence and the trend component is regarded as the *seasonal* component. On top of the decomposition scheme of Autoformer, FEDformer [31] further proposes the mixture of experts’ strategies to mix the trend components extracted by moving average kernels with various kernel sizes.

Input embedding strategies: The self-attention layer in the Transformer architecture cannot preserve the positional information of the time series. However, local positional information, i.e. the ordering of time series, is important. Besides, global temporal information, such as hierarchical timestamps (week, month, year) and agnostic timestamps (holidays and events), is also informative [30]. To enhance the temporal context of time-series inputs, a practical design in the SOTA Transformer-based methods is injecting several embeddings, like a fixed positional encoding, a channel projection embedding, and learnable temporal embeddings into the input sequence. Moreover, temporal embeddings with a temporal convolution layer [16] or learnable timestamps [28] are introduced.

Self-attention schemes: Transformers rely on the self-attention mechanism to extract the semantic dependencies between paired elements. Motivated by reducing the $O(L^2)$ time and memory complexity of the vanilla Transformer, recent works propose two strategies for efficiency. On the one hand, LogTrans and Pyraformer explicitly introduce a sparsity bias into the self-attention scheme. Specifically, LogTrans uses a Logsparse mask to reduce the computational complexity to $O(L \log L)$ while Pyraformer adopts pyramidal attention that captures hierarchically multi-scale temporal dependencies with an $O(L)$ time and memory complexity. On the other hand, Informer and FEDformer use the low-rank property in the self-attention matrix. Informer proposes a ProbSparse self-

attention mechanism and a self-attention distilling operation to decrease the complexity to $O(L \log L)$, and FEDformer designs a Fourier enhanced block and a wavelet enhanced block with random selection to obtain $O(L)$ complexity. Lastly, Autoformer designs a series-wise auto-correlation mechanism to replace the original self-attention layer.

Decoders: The vanilla Transformer decoder outputs sequences in an autoregressive manner, resulting in a slow inference speed and error accumulation effects, especially for long-term predictions. Informer designs a generative-style decoder for DMS forecasting. Other Transformer variants employ similar DMS strategies. For instance, Pyraformer uses a fully-connected layer concatenating Spatio-temporal axes as the decoder. Autoformer sums up two refined decomposed features from trend-cyclical components and the stacked auto-correlation mechanism for seasonal components to get the final prediction. FEDformer also uses a decomposition scheme with the proposed frequency attention block to decode the final results.

The premise of Transformer models is the semantic correlations between paired elements, while the self-attention mechanism itself is permutation-invariant, and its capability of modeling temporal relations largely depends on positional encodings associated with input tokens. Considering the raw numerical data in time series (e.g., stock prices or electricity values), there are hardly any point-wise semantic correlations between them. In time series modeling, we are mainly interested in the temporal relations among a continuous set of points, and the order of these elements instead of the paired relationship plays the most crucial role. While employing positional encoding and using tokens to embed sub-series facilitate preserving some ordering information, the nature of the permutation-invariant self-attention mechanism inevitably results in temporal information loss. Due to the above observations, we are interested in revisiting the effectiveness of Transformer-based LTSF solutions.

4. An Embarrassingly Simple Baseline

In the experiments of existing Transformer-based LTSF solutions ($T \gg 1$), **all the compared (non-Transformer)**

baselines are IMS forecasting techniques, which are known to suffer from significant error accumulation effects. We hypothesize that the performance improvements in these works are largely due to the DMS strategy used in them.

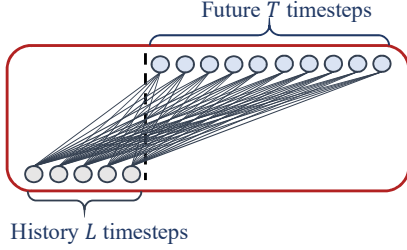


Figure 2. Illustration of the basic linear model.

To validate this hypothesis, we present the simplest DMS model via a temporal linear layer, named *LTSF-Linear*, as a baseline for comparison. The basic formulation of *LTSF-Linear* directly regresses historical time series for future prediction via a weighted sum operation (as illustrated in Figure 2). The mathematical expression is $\hat{X}_i = WX_i$, where $W \in \mathbb{R}^{T \times L}$ is a linear layer along the temporal axis. $\hat{X}_i$ and X_i are the prediction and input for each i_{th} variate. **Note that *LTSF-Linear* shares weights across different variates and does not model any spatial correlations.**

LTSF-Linear is a set of linear models. *Vanilla Linear* is a one-layer linear model. To handle time series across different domains (e.g., finance, traffic, and energy domains), we further introduce two variants with two preprocessing methods, named *DLinear* and *NLinear*.

- Specifically, ***DLinear*** is a combination of a *Decomposition* scheme used in Autoformer and FEDformer with linear layers. It first decomposes a raw data input into a trend component by a moving average kernel and a remainder (seasonal) component. Then, **two one-layer linear layers are applied to each component, and we sum up the two features to get the final prediction.** By explicitly handling trend, *DLinear* enhances the performance of a vanilla linear when there is a clear trend in the data.
- Meanwhile, to boost the performance of *LTSF-Linear* when there is a **distribution shift in the dataset**, *NLinear* first subtracts the input by the last value of the sequence. Then, the input goes through a linear layer, and the subtracted part is added back before making the final prediction. **The subtraction and addition in *NLinear* are a simple normalization for the input sequence.**

5. Experiments

5.1. Experimental Settings

Dataset. We conduct extensive experiments on nine widely-used real-world datasets, including ETT (Electricity Transformer Temperature) [30] (ETTh1, ETTh2, ETTm1, ETTm2), Traffic, Electricity, Weather, ILI, Exchange-Rate [15]. All of them are multivariate time series. We leave *data descriptions* in the Appendix.

Evaluation metric. Following previous works [28, 30, 31], we use Mean Squared Error (MSE) and Mean Absolute Error (MAE) as the core metrics to compare performance.

Compared methods. We include five recent Transformer-based methods: FEDformer [31], Autoformer [28], Informer [30], Pyraformer [18], and LogTrans [16]. Besides, we include a naive DMS method: Closest Repeat (*Repeat*), which repeats the last value in the look-back window, as another simple baseline. Since there are two variants of FEDformer, we compare the one with better accuracy (FEDformer-f via Fourier transform).

5.2. Comparison with Transformers

Quantitative results. In Table 2, we extensively evaluate all mentioned Transformers on nine benchmarks, following the experimental setting of previous work [28, 30, 31]. Surprisingly, the performance of *LTSF-Linear* surpasses the SOTA FEDformer in most cases by 20% ~ 50% improvements on the *multivariate forecasting*, where *LTSF-Linear* even does not model correlations among variates. For different time series benchmarks, *NLinear* and *DLinear* show the superiority to handle the distribution shift and trend-seasonality features. We also provide results for *univariate forecasting* of ETT datasets in the Appendix, where *LTSF-Linear* still consistently outperforms Transformer-based LTSF solutions by a large margin.

FEDformer achieves competitive forecasting accuracy on ETTh1. This because FEDformer employs classical time series analysis techniques such as frequency processing, which brings in time series inductive bias and benefits the ability of temporal feature extraction. In summary, these results reveal that existing complex Transformer-based LTSF solutions are not seemingly effective on the existing nine benchmarks while *LTSF-Linear* can be a powerful baseline.

Another interesting observation is that even though the naive *Repeat* method shows worse results when predicting long-term seasonal data (e.g., Electricity and Traffic), it surprisingly outperforms all Transformer-based methods on Exchange-Rate (around 45%). This is mainly caused by the wrong prediction of trends in Transformer-based solutions, which may overfit toward sudden change noises in the training data, resulting in significant accuracy degradation (see Figure 3(b)). Instead, *Repeat* does not have the bias.

Qualitative results. As shown in Figure 3, we plot

Datasets	ETTh1&ETTh2	ETTm1 & ETTm2	Traffic	Electricity	Exchange-Rate	Weather	ILI
Variates	7	7	862	321	8	21	7
Timesteps	17,420	69,680	17,544	26,304	7,588	52,696	966
Granularity	1hour	5min	1hour	1hour	1day	10min	1week

Table 1. The statistics of the nine popular datasets for the LTSF problem.

Methods	IMP.	Linear*		NLinear*		DLinear*		FEDformer		Autoformer		Informer		Pyraformer*		LogTrans		Repeat*		
Metric	MSE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	
Electricity	96	27.40%	0.140	0.237	0.141	0.237	0.140	0.237	<u>0.193</u>	<u>0.308</u>	0.201	0.317	0.274	0.368	0.386	0.449	0.258	0.357	1.588	0.946
	192	23.88%	0.153	0.250	0.154	0.248	0.153	0.249	<u>0.201</u>	<u>0.315</u>	0.222	0.334	0.296	0.386	0.386	0.443	0.266	0.368	1.595	0.950
	336	21.02%	0.169	0.268	0.171	0.265	0.169	0.267	<u>0.214</u>	<u>0.329</u>	0.231	0.338	0.300	0.394	0.378	0.443	0.280	0.380	1.617	0.961
	720	17.47%	0.203	0.301	0.210	0.297	0.203	0.301	<u>0.246</u>	<u>0.355</u>	0.254	0.361	0.373	0.439	0.376	0.445	0.283	0.376	1.647	0.975
Exchange	96	45.27%	0.082	0.207	0.089	0.208	0.081	0.203	<u>0.148</u>	<u>0.278</u>	0.197	0.323	0.847	0.752	0.376	1.105	0.968	0.812	0.081	0.196
	192	42.06%	0.167	0.304	0.180	0.300	0.157	0.293	<u>0.271</u>	<u>0.380</u>	0.300	0.369	1.204	0.895	1.748	1.151	1.040	0.851	0.167	0.289
	336	33.69%	0.328	0.432	0.331	0.415	0.305	0.414	<u>0.460</u>	<u>0.500</u>	0.509	0.524	1.672	1.036	1.874	1.172	1.659	1.081	0.305	0.396
	720	46.19%	0.964	0.750	1.033	0.780	0.643	0.601	<u>1.195</u>	<u>0.841</u>	1.447	0.941	2.478	1.310	1.943	1.206	1.941	1.127	0.823	0.681
Traffic	96	30.15%	0.410	0.282	0.410	0.279	0.410	0.282	<u>0.587</u>	<u>0.366</u>	0.613	0.388	0.719	0.391	2.085	0.468	0.684	0.384	2.723	1.079
	192	29.96%	0.423	0.287	0.423	0.284	0.423	0.287	<u>0.604</u>	<u>0.373</u>	0.616	0.382	0.696	0.379	0.867	0.467	0.685	0.390	2.756	1.087
	336	29.95%	0.436	0.295	0.435	0.290	0.436	0.296	<u>0.621</u>	<u>0.383</u>	0.622	<u>0.337</u>	0.777	0.420	0.869	0.469	0.734	0.408	2.791	1.095
	720	25.87%	0.466	0.315	0.464	0.307	0.466	0.315	<u>0.626</u>	<u>0.382</u>	0.660	0.408	0.864	0.472	0.881	0.473	0.717	0.396	2.811	1.097
Weather	96	18.89%	0.176	0.236	0.182	0.232	0.176	0.237	<u>0.217</u>	<u>0.296</u>	0.266	0.336	0.300	0.384	0.896	0.556	0.458	0.490	0.259	0.254
	192	21.01%	0.218	0.276	0.225	0.269	0.220	0.282	<u>0.276</u>	<u>0.336</u>	0.307	0.367	0.598	0.544	0.622	0.624	0.658	0.589	0.309	0.292
	336	22.71%	0.262	0.312	0.271	0.301	0.265	0.319	<u>0.339</u>	<u>0.380</u>	0.359	0.395	0.578	0.523	0.739	0.753	0.797	0.652	0.377	0.338
	720	19.85%	0.326	0.365	0.338	0.348	0.323	0.362	<u>0.403</u>	<u>0.428</u>	0.419	0.428	1.059	0.741	1.004	0.934	0.869	0.675	0.465	0.394
ILI	24	47.86%	1.947	0.985	1.683	0.858	2.215	1.081	<u>3.228</u>	<u>1.260</u>	3.483	1.287	5.764	1.677	1.420	2.012	4.480	1.444	6.587	1.701
	36	36.43%	2.182	1.036	1.703	0.859	1.963	0.963	<u>2.679</u>	<u>1.080</u>	3.103	1.148	4.755	1.467	7.394	2.031	4.799	1.467	7.130	1.884
	48	34.43%	2.256	1.060	1.719	0.884	2.130	1.024	<u>2.622</u>	<u>1.078</u>	2.669	1.085	4.763	1.469	7.551	2.057	4.800	1.468	6.575	1.798
	60	34.33%	2.390	1.104	1.819	0.917	2.368	1.096	<u>2.857</u>	<u>1.157</u>	<u>2.770</u>	<u>1.125</u>	5.264	1.564	7.662	2.100	5.278	1.560	5.893	1.677
ETTh1	96	0.80%	0.375	0.397	0.374	0.394	0.375	0.399	<u>0.376</u>	<u>0.419</u>	0.449	0.459	0.865	0.713	0.664	0.612	0.878	0.740	1.295	0.713
	192	3.57%	0.418	0.429	0.408	0.415	0.405	0.416	<u>0.420</u>	<u>0.448</u>	0.500	0.482	1.008	0.792	0.790	0.681	1.037	0.824	1.325	0.733
	336	6.54%	0.479	0.476	0.429	0.427	0.439	0.443	<u>0.459</u>	<u>0.465</u>	0.521	0.496	1.107	0.809	0.891	0.738	1.238	0.932	1.323	0.744
	720	13.04%	0.624	0.592	0.440	0.453	0.472	0.490	<u>0.506</u>	<u>0.507</u>	0.514	0.512	1.181	0.865	0.963	0.782	1.135	0.852	1.339	0.756
ETTh2	96	19.94%	0.288	0.352	0.277	0.338	0.289	0.353	<u>0.346</u>	<u>0.388</u>	0.358	0.397	3.755	1.525	0.645	0.597	2.116	1.197	0.432	0.422
	192	19.81%	0.377	0.413	0.344	0.381	0.383	0.418	<u>0.429</u>	<u>0.439</u>	0.456	0.452	5.602	1.931	0.788	0.683	4.315	1.635	0.534	0.473
	336	25.93%	0.452	0.461	0.357	0.400	0.448	0.465	<u>0.496</u>	<u>0.487</u>	0.482	0.486	4.721	1.835	0.907	0.747	1.124	1.604	0.591	0.508
	720	14.25%	0.698	0.595	0.394	0.436	0.605	0.551	<u>0.463</u>	<u>0.474</u>	0.515	0.511	3.647	1.625	0.963	0.783	3.188	1.540	0.588	0.517
ETTm1	96	21.10%	0.308	0.352	0.306	0.348	0.299	0.343	<u>0.379</u>	<u>0.419</u>	0.505	0.475	0.672	0.571	0.543	0.510	0.600	0.546	1.214	0.665
	192	21.36%	0.340	0.369	0.349	0.375	0.335	0.365	<u>0.426</u>	<u>0.441</u>	0.553	0.496	0.795	0.669	0.557	0.537	0.837	0.700	1.261	0.690
	336	17.07%	0.376	0.393	0.375	0.388	0.369	0.386	<u>0.445</u>	<u>0.459</u>	0.621	0.537	1.212	0.871	0.754	0.655	1.124	0.832	1.283	0.707
	720	21.73%	0.440	0.435	0.433	0.422	0.425	0.421	<u>0.543</u>	<u>0.490</u>	0.671	0.561	1.166	0.823	0.908	0.724	1.153	0.820	1.319	0.729
ETTm2	96	17.73%	0.168	0.262	0.167	0.255	0.167	0.260	<u>0.203</u>	<u>0.287</u>	0.255	0.339	0.365	0.453	0.435	0.507	0.768	0.642	0.266	0.328
	192	17.84%	0.232	0.308	0.221	0.293	0.224	0.303	<u>0.269</u>	<u>0.328</u>	0.281	0.340	0.533	0.563	0.730	0.673	0.989	0.757	0.340	0.371
	336	15.69%	0.320	0.373	0.274	0.327	0.281	0.342	<u>0.325</u>	<u>0.366</u>	0.339	0.372	1.363	0.887	1.201	0.845	1.334	0.872	0.412	0.410
	720	12.58%	0.413	0.435	0.368	0.384	0.397	0.421	<u>0.421</u>	<u>0.415</u>	0.433	0.432	3.379	1.338	3.625	1.451	3.048	1.328	0.521	0.465

- Methods* are implemented by us; Other results are from FEDformer [31].

Table 2. Multivariate long-term forecasting errors in terms of MSE and MAE, the lower the better. Among them, ILI dataset is with forecasting horizon $T \in \{24, 36, 48, 60\}$. For the others, $T \in \{96, 192, 336, 720\}$. Repeat repeats the last value in the look-back window. The best results are highlighted in bold and the best results of Transformers are highlighted with a underline. Accordingly, IMP is the best result of linear models compared to the results of Transformer-based solutions.

the prediction results on three selected time series datasets with Transformer-based solutions and *LTSF-Linear*: Electricity (Sequence 1951, Variate 36), Exchange-Rate (Sequence 676, Variate 3), and ETTh2 (Sequence 1241, Variate 2), where these datasets have different temporal patterns. When the input length is 96 steps, and the output horizon is 336 steps, Transformers [28, 30, 31] fail to capture the scale and bias of the future data on Electricity and ETTh2. Moreover, they can hardly predict a proper trend on aperiodic data such as Exchange-Rate. These phenomena further indicate the inadequacy of existing Transformer-based solutions for the LTSF task.

5.3. More Analyses on LTSF-Transformers

Can existing LTSF-Transformers extract temporal relations well from longer input sequences? The size of the look-back window greatly impacts forecasting accuracy as

it determines how much we can learn from historical data. Generally speaking, a powerful TSF model with a strong temporal relation extraction capability should be able to achieve better results with larger look-back window sizes.

To study the impact of input look-back window sizes, we conduct experiments with $L \in \{24, 48, 72, 96, 120, 144, 168, 192, 336, 504, 672, 720\}$ for long-term forecasting ($T=720$). Figure 4 demonstrates the MSE results on two datasets. Similar to the observations from previous studies [27, 30], existing Transformer-based models' performance deteriorates or stays stable when the look-back window size increases. In contrast, the performances of all *LTSF-Linear* are significantly boosted with the increase of look-back window size. Thus, existing solutions tend to overfit temporal noises instead of extracting temporal information if given a longer sequence, and the input size 96 is exactly suitable for most Transformers.

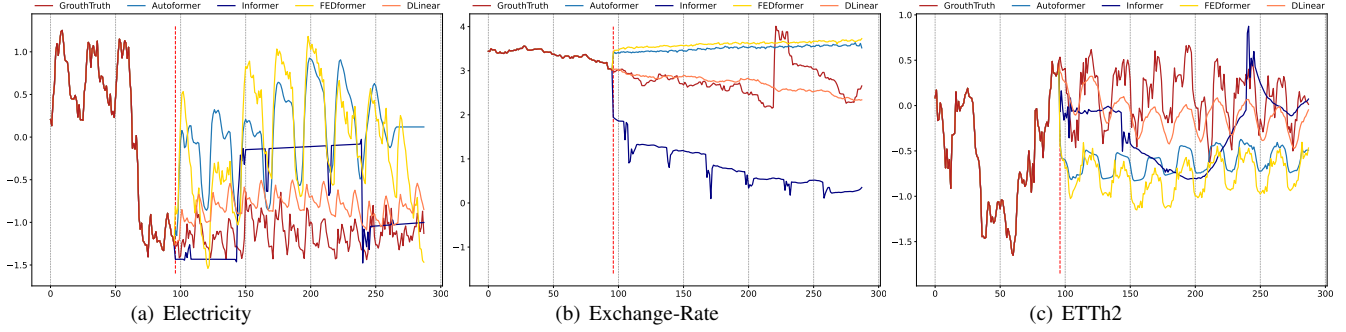


Figure 3. Illustration of the long-term forecasting output (Y-axis) of five models with an input length $L=96$ and output length $T=192$ (X-axis) on Electricity, Exchange-Rate, and ETTh2, respectively.

Additionally, we provide more quantitative results in the Appendix, and our conclusion holds in almost all cases.

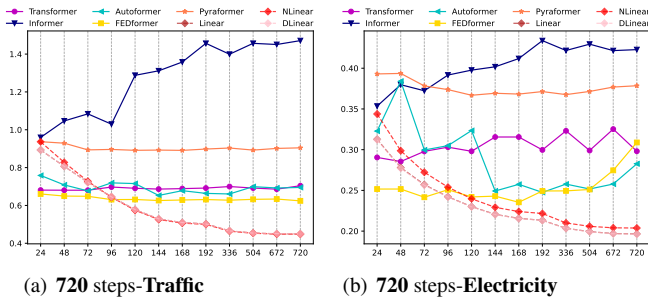


Figure 4. The MSE results (Y-axis) of models with different look-back window sizes (X-axis) of long-term forecasting ($T=720$) on the Traffic and Electricity datasets.

What can be learned for long-term forecasting? While the temporal dynamics in the look-back window significantly impact the forecasting accuracy of short-term time series forecasting, we hypothesize that long-term forecasting depends on whether *models can capture the trend and periodicity well only*. That is, the farther the forecasting horizon, the less impact the look-back window itself has.

Methods	FEDformer		Autoformer	
Input	<i>Close</i>	<i>Far</i>	<i>Close</i>	<i>Far</i>
Electricity	0.251	0.265	0.255	0.287
Traffic	0.631	0.645	0.677	0.675

Table 3. Comparison of different input sequences under the MSE metric to explore what LTSF-Transformers depend on. If the input is *Close*, we use the $96_{th}, \dots, 191_{th}$ time steps as the input sequence. If the input is *Far*, we use the $0_{th}, \dots, 95_{th}$ time steps. Both of them forecast the $192_{th}, \dots, (192 + 720)_{th}$ time steps.

To validate the above hypothesis, in Table 3, we compare the forecasting accuracy for the same future 720 time steps with data from two different look-back windows: (i). the original input $L=96$ setting (called *Close*) and (ii). the far input $L=96$ setting (called *Far*) that is before the original

96 time steps. From the experimental results, the performance of the SOTA Transformers drops slightly, indicating these models only capture similar temporal information from the adjacent time series sequence. Since capturing the intrinsic characteristics of the dataset generally does not require a large number of parameters, i.e. one parameter can represent the periodicity. Using too many parameters will even cause overfitting, which partially explains why *LTSF-Linear* performs better than Transformer-based methods.

Are the self-attention scheme effective for LTSF? We verify whether these complex designs in the existing Transformer (e.g., Informer) are essential. In Table 4, we gradually transform Informer to Linear. First, we replace each self-attention layer by a linear layer, called *Att.-Linear*, since a self-attention layer can be regarded as a fully-connected layer where weights are dynamically changed. Furthermore, we discard other auxiliary designs (e.g., FFN) in Informer to leave embedding layers and linear layers, named *Embed + Linear*. Finally, we simplify the model to one linear layer. Surprisingly, the performance of Informer grows with the gradual simplification, indicating the unnecessary of the self-attention scheme and other complex modules at least for existing LTSF benchmarks.

Methods	Informer	<i>Att.-Linear</i>	<i>Embed + Linear</i>	Linear
Exchange	96	0.847	1.003	0.173
	192	1.204	0.979	0.443
	336	1.672	1.498	1.288
	720	2.478	2.102	0.763
ETTh1	96	0.865	0.613	0.454
	192	1.008	0.759	0.686
	336	1.107	0.921	0.821
	720	1.181	0.902	1.051

Table 4. The MSE comparisons of gradually transforming Informer to a Linear from the left to right columns. *Att.-Linear* is a structure that replaces each attention layer with a linear layer. *Embed + Linear* is to drop other designs and only keeps embedding layers and a linear layer. The look-back window size is 96.

Can existing LTSF-Transformers preserve temporal order well? Self-attention is inherently permutation-

Methods		Linear			FEDformer			Autoformer			Informer		
Predict Length		<i>Ori.</i>	<i>Shuf.</i>	<i>Half-Ex.</i>	<i>Ori.</i>	<i>Shuf.</i>	<i>Half-Ex.</i>	<i>Ori.</i>	<i>Shuf.</i>	<i>Half-Ex.</i>	<i>Ori.</i>	<i>Shuf.</i>	<i>Half-Ex.</i>
Exchange	96	0.080	0.133	0.169	0.161	0.160	0.162	0.152	0.158	0.160	0.952	1.004	0.959
	192	0.162	0.208	0.243	0.274	0.275	0.275	0.278	0.271	0.277	1.012	1.023	1.014
	336	0.286	0.320	0.345	0.439	0.439	0.439	0.435	0.430	0.435	1.177	1.181	1.177
	720	0.806	0.819	0.836	1.122	1.122	1.122	1.113	1.113	1.113	1.198	1.210	1.196
Average Drop		N/A	27.26%	46.81%	N/A	-0.09%	0.20%	N/A	0.09%	1.12%	N/A	-0.12%	-0.18%
ETTh1	96	0.395	0.824	0.431	0.376	0.753	0.405	0.455	0.838	0.458	0.974	0.971	0.971
	192	0.447	0.824	0.471	0.419	0.730	0.436	0.486	0.774	0.491	1.233	1.232	1.231
	336	0.490	0.825	0.505	0.447	0.736	0.453	0.496	0.752	0.497	1.693	1.693	1.691
	720	0.520	0.846	0.528	0.468	0.720	0.470	0.525	0.696	0.524	2.720	2.716	2.715
Average Drop		N/A	81.06%	4.78%	N/A	73.28%	3.44%	N/A	56.91%	0.46%	N/A	1.98%	0.18%

Table 5. The MSE comparisons of models when shuffling the raw input sequence. *Shuf.* randomly shuffles the input sequence. *Half-EX.* randomly exchanges the first half of the input sequences with the second half. Average Drop is the average performance drop under all forecasting lengths after shuffling. All results are the average test MSE of five runs.

invariant, i.e., regardless of the order. However, in time-series forecasting, the sequence order often plays a crucial role. We argue that even with positional and temporal embeddings, existing Transformer-based methods still suffer from temporal information loss. In Table 5, we shuffle the raw input before the embedding strategies. Two shuffling strategies are presented: *Shuf.* randomly shuffles the whole input sequences and *Half-Ex.* exchanges the first half of the input sequence with the second half. Interestingly, compared with the original setting (*Ori.*) on the Exchange Rate, the performance of all Transformer-based methods does not fluctuate even when the input sequence is randomly shuffled. By contrary, the performance of *LTSF-Linear* is damaged significantly. These indicate that *LTSF-Transformers* with different positional and temporal embeddings preserve quite limited temporal relations and are prone to overfit on noisy financial data, while the *LTSF-Linear* can model the order naturally and avoid overfitting with fewer parameters.

For the ETTh1 dataset, FEDformer and Autoformer introduce time series inductive bias into their models, making them can extract certain temporal information when the dataset has more clear temporal patterns (e.g., periodicity) than the Exchange Rate. Therefore, the average drops of the two Transformers are 73.28% and 56.91% under the *Shuf.* setting, where it loses the whole order information. Moreover, Informer still suffers less from both *Shuf.* and *Half-Ex.* settings due to its no such temporal inductive bias. Overall, the average drops of *LTSF-Linear* are larger than Transformer-based methods for all cases, indicating the existing Transformers do not preserve temporal order well.

How effective are different embedding strategies? We study the benefits of position and timestamp embeddings used in Transformer-based methods. In Table 6, the forecasting errors of Informer largely increase without positional embeddings (wo/Pos.). Without timestamp embeddings (wo/Temp.) will gradually damage the performance of Informer as the forecasting lengths increase. Since Informer uses a single time step for each token, it is necessary to introduce temporal information in tokens.

Methods	Embedding	Traffic			
		96	192	336	720
FEDformer	All	0.597	0.606	0.627	0.649
	wo/Pos.	0.587	0.604	0.621	0.626
	wo/Temp.	0.613	0.623	0.650	0.677
	wo/Pos.-Temp.	0.613	0.622	0.648	0.663
Autoformer	All	0.629	0.647	0.676	0.638
	wo/Pos.	0.613	0.616	0.622	0.660
	wo/Temp.	0.681	0.665	0.908	0.769
	wo/Pos.-Temp.	0.672	0.811	1.133	1.300
Informer	All	0.719	0.696	0.777	0.864
	wo/Pos.	1.035	1.186	1.307	1.472
	wo/Temp.	0.754	0.780	0.903	1.259
	wo/Pos.-Temp.	1.038	1.351	1.491	1.512

Table 6. The MSE comparisons of different embedding strategies on Transformer-based methods with look-back window size 96 and forecasting lengths {96, 192, 336, 720}.

Rather than using a single time step in each token, FEDformer and Autoformer input a sequence of timestamps to embed the temporal information. Hence, they can achieve comparable or even better performance without fixed positional embeddings. However, without timestamp embeddings, the performance of Autoformer declines rapidly because of the loss of global temporal information. Instead, thanks to the frequency-enhanced module proposed in FEDformer to introduce temporal inductive bias, it suffers less from removing any position/timestamp embeddings.

Is training data size a limiting factor for existing LTSF-Transformers? Some may argue that the poor performance of Transformer-based solutions is due to the small sizes of the benchmark datasets. Unlike computer vision or natural language processing tasks, TSF is performed on collected time series, and it is difficult to scale up the training data size. In fact, the size of the training data would indeed have a significant impact on the model performance. Accordingly, we conduct experiments on Traffic, comparing the performance of the model trained on a full dataset (17,544*0.7 hours), named *Ori.*, with that trained on a shortened dataset (8,760 hours, i.e., 1 year), called *Short*. Unexpectedly, Table 7 presents that the prediction errors

with reduced training data are lower in most cases. This might be because the whole-year data maintains more clear temporal features than a longer but incomplete data size. While we cannot conclude that we should use less data for training, it demonstrates that the training data scale is not the limiting reason for the performances of Autoformer and FEDformer.

Methods	FEDformer		Autoformer	
Dataset	<i>Ori.</i>	<i>Short</i>	<i>Ori.</i>	<i>Short</i>
96	0.587	0.568	0.613	0.594
192	0.604	0.584	0.616	0.621
336	0.621	0.601	0.622	0.621
720	0.626	0.608	0.660	0.650

Table 7. The MSE comparison of two training data sizes.

Is efficiency really a top-level priority? Existing LTSF-Transformers claim that the $O(L^2)$ complexity of the vanilla Transformer is unaffordable for the LTSF problem. Although they prove to be able to improve the theoretical time and memory complexity from $O(L^2)$ to $O(L)$, it is unclear whether 1) the actual inference time and memory cost on devices are improved, and 2) the memory issue is unacceptable and urgent for today’s GPU (e.g., an NVIDIA Titan XP here). In Table 8, we compare the average practical efficiencies with 5 runs. Interestingly, compared with the vanilla Transformer (with the same DMS decoder), most Transformer variants incur similar or even worse inference time and parameters in practice. These follow-ups introduce more additional design elements to make practical costs high. Moreover, the memory cost of the vanilla Transformer is practically acceptable, even for output length $L = 720$, which weakens the importance of developing a memory-efficient Transformers, at least for existing benchmarks.

Method	MACs	Parameter	Time	Memory
DLinear	0.04G	139.7K	0.4ms	687MiB
Transformer×	4.03G	13.61M	26.8ms	6091MiB
Informer	3.93G	14.39M	49.3ms	3869MiB
Autoformer	4.41G	14.91M	164.1ms	7607MiB
Pyraformer	0.80G	241.4M*	3.4ms	7017MiB
FEDformer	4.41G	20.68M	40.5ms	4143MiB

× is modified into the same one-step decoder, which is implemented in the source code from Autoformer.
 * 236.7M parameters of Pyraformer come from its linear decoder.

Table 8. Comparison of practical efficiency of LTSF-Transformers under $L=96$ and $T=720$ on the Electricity. MACs are the number of multiply-accumulate operations. We use Dlinear for comparison since it has the double cost in *LTSF-Linear*. The inference time averages 5 runs.

6. Conclusion and Future Work

Conclusion. This work questions the effectiveness of emerging favored Transformer-based solutions for the long-term time series forecasting problem. We use an embarrassingly simple linear model *LTSF-Linear* as a DMS forecasting baseline to verify our claims. Note that our

contributions do not come from proposing a linear model but rather from throwing out an important question, showing surprising comparisons, and demonstrating why LTSF-Transformers are not as effective as claimed in these works through various perspectives. We sincerely hope our comprehensive studies can benefit future work in this area.

Future work. *LTSF-Linear* has a limited model capacity, and it merely serves a simple yet competitive baseline with strong interpretability for future research. For example, the one-layer linear network is hard to capture the temporal dynamics caused by change points [25]. Consequently, we believe there is a great potential for new model designs, data processing, and benchmarks to tackle the challenging LTSF problem.

Appendix: Are Transformers Effective for Time Series Forecasting?

In this Appendix, we provide descriptions of non-Transformer-based TSF solutions, detailed experimental settings, more comparisons under different look-back window sizes, and the visualization of *LTSF-Linear* on all datasets. We also append our code to reproduce the results shown in the paper.

A. Related Work: Non-Transformer-Based TSF Solutions

As a long-standing problem with a wide range of applications, statistical approaches (e.g., autoregressive integrated moving average (ARIMA) [1], exponential smoothing [12], and structural models [14]) for time series forecasting have been used from the 1970s onward. Generally speaking, the parametric models used in statistical methods require significant domain expertise to build.

To relieve this burden, many machine learning techniques such as gradient boosting regression tree (GBRT) [10, 11] gain popularity, which learns the temporal dynamics of time series in a data-driven manner. However, these methods still require manual feature engineering and model designs. With the powerful representation learning capability of deep neural networks (DNNs) from abundant data, various deep learning-based TSF solutions are proposed in the literature, achieving better forecasting accuracy than traditional techniques in many cases.

Besides Transformers, the other two popular DNN architectures are also applied for time series forecasting:

- Recurrent neural networks (RNNs) based methods (e.g., [21]) summarize the past information compactly in internal memory states and recursively update themselves for forecasting.
- Convolutional neural networks (CNNs) based methods (e.g., [3]), wherein convolutional filters are used to capture local temporal features.

RNN-based TSF methods belong to IMS forecasting techniques. Depending on whether the decoder is implemented in an autoregressive manner, there are either IMS or DMS forecasting techniques for CNN-based TSF methods [3, 17].

B. Experimental Details

B.1. Data Descriptions

We use nine widely-used datasets in the main paper. The details are listed in the following.

- ETT (Electricity Transformer Temperature) [30]² consists of two hourly-level datasets (ETTh) and two 15-minute-level datasets (ETTm). Each of them contains seven oil and load features of electricity transformers from July 2016 to July 2018.
- Traffic³ describes the road occupancy rates. It contains the hourly data recorded by the sensors of San Francisco freeways from 2015 to 2016.
- Electricity⁴ collects the hourly electricity consumption of 321 clients from 2012 to 2014.
- Exchange-Rate [15]⁵ collects the daily exchange rates of 8 countries from 1990 to 2016.
- Weather⁶ includes 21 indicators of weather, such as air temperature, and humidity. Its data is recorded every 10 min for 2020 in Germany.
- ILI⁷ describes the ratio of patients seen with influenza-like illness and the number of patients. It includes weekly data from the Centers for Disease Control and Prevention of the United States from 2002 to 2021.

B.2. Implementation Details

For existing Transformer-based TSF solutions: the implementation of Autoformer [28], Informer [30], and the vanilla Transformer [26] are all taken from the Autoformer work [28]; the implementation of FEDformer [31] and Pyraformer [18] are from their respective code repository. We also adopt their default hyper-parameters to train the models. For *DLinear*, the moving average kernel size for decomposition is 25, which is the same as Autoformer. The total parameters of a vanilla linear model and a *NLinear* are TL. The total parameters of the *DLinear* are 2TL. Since *LTSF-Linear* will be underfitting when the input length is short, and LTSF-Transformers tend to overfit on a long lookback window size. To compare the best performance of existing LTSF-Transformers with *LTSF-Linear*, we report L=336 for *LTSF-Linear* and L=96 for Transformers by default. For more hyper-parameters of *LTSF-Linear*, please refer to our code.

²<https://github.com/zhouhaoyi/ETDataset>

³<http://pems.dot.ca.gov>

⁴<https://archive.ics.uci.edu/ml/datasets/ElectricityLoadDiagrams20112014>

⁵<https://github.com/laiguokun/multivariate-time-series-data>

⁶<https://www.bgc-jena.mpg.de/wetter/>

⁷<https://gis.cdc.gov/grasp/fluview/fluportaldashboard.html>

C. Additional Comparison with Transformers

We further compare *LTSF-Linear* with LTSF-Transformer for Univariate Forecasting on four ETT datasets. Moreover, in Figure 4 of the main paper, we demonstrate that existing Transformers fail to exploit large look-back window sizes with two examples. Here, we give comprehensive comparisons between *LTSF-Linear* and Transformer-based TSF solutions under various look-back window sizes *on all benchmarks*.

C.1. Comparison of Univariate Forecasting

We present the univariate forecasting results on the four ETT datasets in table 9. Similarly, *LTSF-Linear*, especially for *NLinear* can consistently outperform all transformer-based methods by a large margin in most time. We find that there are serious distribution shifts between training and test sets (as shown in Fig. 5 (a), (b)) on ETTh1 and ETTh2 datasets. Simply normalization via the last value from the lookback window can greatly relieve the distribution shift problem.

C.2. Comparison under Different Look-back Windows

In Figure 6, we provide the MSE comparisons of five LTSF-Transformers with *LTSF-Linear* under different look-back window sizes to explore whether existing Transformers can extract temporal well from longer input sequences. For hourly granularity datasets (ETTh1, ETTh2, Traffic, and Electricity), the increasing look-back window sizes are {24, 48, 72, 96, 120, 144, 168, 192, 336, 504, 672, 720}, which represent {1, 2, 3, 4, 5, 6, 7, 8, 14, 21, 28, 30} days. The forecasting steps are {24, 720}, which mean {1, 30} days. For 5-minute granularity datasets (ETTm1 and ETTm2), we set the look-back window size as {24, 36, 48, 60, 72, 144, 288}, which represent {2, 3, 4, 5, 6, 12, 24} hours. For 10-minute granularity datasets (Weather), we set the look-back window size as {24, 48, 72, 96, 120, 144, 168, 192, 336, 504, 672, 720}, which mean {4, 8, 12, 16, 20, 24, 28, 32, 56, 84, 112, 120} hours. The forecasting steps are {24, 720} that are {4, 120} hours. For weekly granularity dataset (ILI), we set the look-back window size as {26, 52, 78, 104, 130, 156, 208}, which represent {0.5, 1, 1.5, 2, 2.5, 3, 3.5, 4} years. The corresponding forecasting steps are {26, 208}, meaning {0.5, 4} years.

As shown in Figure 6, with increased look-back window sizes, the performance of *LTSF-Linear* is significantly boosted for most datasets (e.g., ETTm1 and Traffic), while this is not the case for Transformer-based TSF solutions. Most of their performance fluctuates or gets worse as the input lengths increase. To be specific, the results of Exchange-Rate do not show improved results with a long look-back window (from Figure 6(m) and (n)), and we at-

tribute it to the low information-to-noise ratio in such financial data.

D. Ablation study on the *LTSF-Linear*

D.1. Motivation of *NLinear*

If we normalize the test data by the mean and variance of train data, there could be a distribution shift in testing data, i.e, the mean value of testing data is not 0. If the model made a prediction that is out of the distribution of true value, a large error would occur. For example, there is a large error between the true value and the true value minus/add one. Therefore, in *NLinear*, we use the subtraction and addition to shift the model prediction toward the distribution of true value. Then, large errors are avoided, and the model performances can be improved. Figure 5 illustrates histograms of the trainset-test set distributions, where each bar represents the number of data points. Clear distribution shifts between training and testing data can be observed in ETTh1, ETTh2, and ILI. Accordingly, from Table 9 and Table 2 in the main paper, we can observe that there are great improvements in the three datasets comparing the *NLinear* to the *Linear*, showing the effectiveness of the *NLinear* in relieving distribution shifts. Moreover, for the datasets without obvious distribution shifts, like Electricity in Figure 5(c), using the vanilla *Linear* can be enough, demonstrating the similar performance with *NLinear* and *DLinear*.

D.2. The Features of *LTSF-Linear*

Although *LTSF-Linear* is simple, it has some compelling characteristics:

- **An $O(1)$ maximum signal traversing path length:** The shorter the path, the better the dependencies are captured [18], making *LTSF-Linear* capable of capturing both short-range and long-range temporal relations.
- **High-efficiency:** As *LTSF-Linear* is a linear model with two linear layers at most, it costs much lower memory and fewer parameters and has a faster inference speed than existing Transformers (see Table 8 in main paper).
- **Interpretability:** After training, we can visualize weights from the seasonality and trend branches to have some insights on the predicted values [9].
- **Easy-to-use:** *LTSF-Linear* can be obtained easily without tuning model hyper-parameters.

D.3. Interpretability of *LTSF-Linear*

Because *LTSF-Linear* is a set of linear models, the weights of linear layers can directly reveal how *LTSF-Linear* works. The weight visualization of *LTSF-Linear* can

Methods		Linear		NLinear		DLinear		FEDformer-f		FEDformer-w		Autoformer		Informer		LogTrans	
Metric		MSE	MAE	MSE	MSE	MAE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
ETTh1	96	0.189	0.359	0.053	0.177	0.056	0.180	0.079	0.215	0.080	0.214	<u>0.071</u>	<u>0.206</u>	0.193	0.377	0.283	0.468
	192	0.078	0.212	0.069	0.204	0.071	0.204	<u>0.104</u>	<u>0.245</u>	0.105	0.256	0.114	0.262	0.217	0.395	0.234	0.409
	336	0.091	0.237	0.081	0.226	0.098	0.244	0.119	0.270	0.120	0.269	<u>0.107</u>	<u>0.258</u>	0.202	0.381	0.386	0.546
	720	0.172	0.340	0.080	0.226	0.189	0.359	0.142	0.299	0.127	0.280	<u>0.126</u>	<u>0.283</u>	0.183	0.355	0.475	0.629
ETTh2	96	0.133	0.283	0.129	0.278	0.131	0.279	<u>0.128</u>	<u>0.271</u>	0.156	0.306	0.153	0.306	0.213	0.373	0.217	0.379
	192	0.176	0.330	0.169	0.324	0.176	0.329	<u>0.185</u>	<u>0.330</u>	0.238	0.380	0.204	0.351	0.227	0.387	0.281	0.429
	336	0.213	0.371	0.194	0.355	0.209	0.367	<u>0.231</u>	<u>0.378</u>	0.271	0.412	0.246	0.389	0.242	0.401	0.293	0.437
	720	0.292	0.440	0.225	0.381	0.276	0.426	0.278	0.420	0.288	0.438	<u>0.268</u>	<u>0.409</u>	0.291	0.439	0.218	0.387
ETTm1	96	0.028	0.125	0.026	0.122	0.028	0.123	<u>0.033</u>	<u>0.140</u>	0.036	0.149	0.056	0.183	0.109	0.277	0.049	0.171
	192	0.043	0.154	0.039	0.149	0.045	0.156	<u>0.058</u>	<u>0.186</u>	0.069	0.206	0.081	0.216	0.151	0.310	0.157	0.317
	336	0.059	0.180	0.052	0.172	0.061	0.182	0.084	0.231	<u>0.071</u>	<u>0.209</u>	0.076	0.218	0.427	0.591	0.289	0.459
	720	0.080	0.211	0.073	0.207	0.080	0.210	<u>0.102</u>	0.250	0.105	<u>0.248</u>	0.110	0.267	0.438	0.586	0.430	0.579
ETTm2	96	0.066	0.189	0.063	0.182	0.063	0.183	0.067	0.198	<u>0.063</u>	<u>0.189</u>	0.065	0.189	0.088	0.225	0.075	0.208
	192	0.094	0.230	0.090	0.223	0.092	0.227	<u>0.102</u>	<u>0.245</u>	0.110	0.252	0.118	0.256	0.132	0.283	0.129	0.275
	336	0.120	0.263	0.117	0.259	0.119	0.261	<u>0.130</u>	<u>0.279</u>	0.147	0.301	0.154	0.305	0.180	0.336	0.154	0.302
	720	0.175	0.320	0.170	0.318	0.175	0.320	<u>0.178</u>	<u>0.325</u>	0.219	0.368	0.182	0.335	0.300	0.435	0.160	0.321

Table 9. Univariate long sequence time-series forecasting results on ETT full benchmark. The **best results** are highlighted in **bold** and the best results of Transformers are highlighted with a underline.

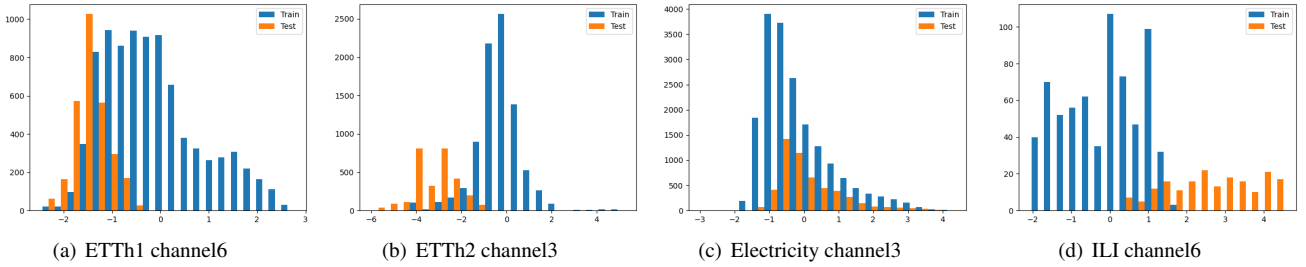


Figure 5. Distribution of ETTh1, ETTh2, Electricity, and ILI dataset. A clear distribution shift between training and testing data can be observed in ETTh1, ETTh2, and ILI.

also reveal certain characteristics in the data used for forecasting.

Here we take DLinear as an example. Accordingly, we visualize the trend and remainder weights of all datasets with a fixed input length of 96 and four different forecasting horizons. To obtain a smooth weight with a clear pattern in visualization, we initialize the weights of the linear layers in DLinear as $1/L$ rather than random initialization. That is, we use the same weight for every forecasting time step in the look-back window at the start of training.

How the model works: Figure 7(c) visualize the weights of the trend and the remaining layers on the Exchange-Rate dataset. Due to the lack of periodicity and seasonality in financial data, it is hard to observe clear patterns, but the trend layer reveals greater weights of information closer to the outputs, representing their larger contributions to the predicted values.

Periodicity of data: For Traffic data, as shown in Figure 7(d), the model gives high weights to the latest time step of the look-back window for the 0,23,47...719 forecast-

ing steps. Among these forecasting time steps, the 0, 167, 335, 503, 671 time steps have higher weights. Note that 24 time steps are a day, and 168 time steps are a week. This indicates that Traffic has a daily periodicity and a weekly periodicity.

References

- [1] Adebisi A Ariyo, Adewumi O Adewumi, and Charles K Ayo. Stock price prediction using the arima model. In *2014 UKSim-AMSS 16th International Conference on Computer Modelling and Simulation*, pages 106–112. IEEE, 2014. 1, 2, 9
- [2] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. *arXiv: Computation and Language*, 2014. 2
- [3] Shaojie Bai, J Zico Kolter, and Vladlen Koltun. An empirical evaluation of generic convolutional and

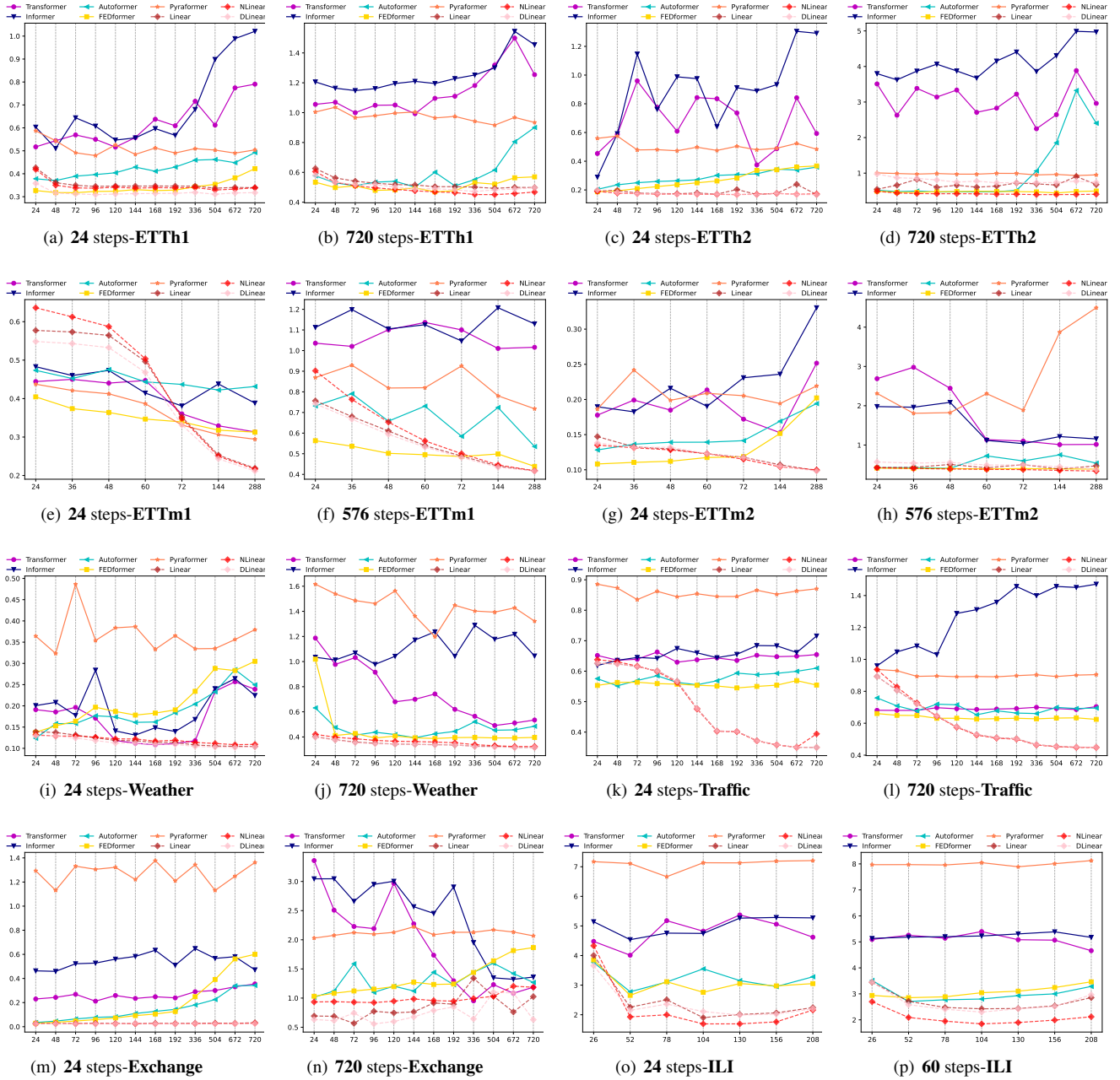


Figure 6. The MSE results (Y-axis) of models with different look-back window sizes (X-axis) of the long-term forecasting (e.g., 720-time steps) and the short-term forecasting (e.g., 24 time steps) on different benchmarks.

recurrent networks for sequence modeling. *arXiv preprint arXiv:1803.01271*, 2018. 1, 9

[4] Guillaume Chevillon. Direct multi-step estimation and forecasting. *Journal of Economic Surveys*, 21(4):746–785, 2007. 2

[5] Razvan-Gabriel Cirstea, Chenjuan Guo, Bin Yang, Tung Kieu, Xuanyi Dong, and Shirui Pan. Triformer:

Triangular, variable-specific attentions for long sequence multivariate time series forecasting—full version. *arXiv preprint arXiv:2204.13767*, 2022. 1

[6] R. B. Cleveland. Stl : A seasonal-trend decomposition procedure based on loess. *Journal of Office Statistics*, 1990. 3

[7] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and

preprint *arXiv:1810.04805*, 2018. 1

- [8] Linhao Dong, Shuang Xu, and Bo Xu. Speech-transformer: a no-recurrence sequence-to-sequence model for speech recognition. In *2018 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 5884–5888. IEEE, 2018. 1
- [9] Ruijun Dong and Witold Pedrycz. A granular time series approach to long-term forecasting and trend forecasting. *Physica A: Statistical Mechanics and its Applications*, 387(13):3253–3270, 2008. 10
- [10] Shereen Elsayed, Daniela Thyssens, Ahmed Rashed, Hadi Samer Jomaa, and Lars Schmidt-Thieme. Do we really need deep learning models for time series forecasting? *arXiv preprint arXiv:2101.02118*, 2021. 9
- [11] Jerome H Friedman. Greedy function approximation: a gradient boosting machine. *Annals of statistics*, pages 1189–1232, 2001. 1, 9
- [12] Everette S Gardner Jr. Exponential smoothing: The state of the art. *Journal of forecasting*, 4(1):1–28, 1985. 9
- [13] James Douglas Hamilton. *Time series analysis*. Princeton university press, 2020. 3
- [14] Andrew C Harvey. Forecasting, structural time series models and the kalman filter. 1990. 9
- [15] Guokun Lai, Wei-Cheng Chang, Yiming Yang, and Hanxiao Liu. Modeling long- and short-term temporal patterns with deep neural networks. *international acm sigir conference on research and development in information retrieval*, 2017. 1, 4, 9
- [16] Shiyang Li, Xiaoyong Jin, Yao Xuan, Xiyu Zhou, Wenhui Chen, Yu-Xiang Wang, and Xifeng Yan. Enhancing the locality and breaking the memory bottleneck of transformer on time series forecasting. *Advances in Neural Information Processing Systems*, 32, 2019. 1, 2, 3, 4
- [17] Minhao Liu, Ailing Zeng, Zhijian Xu, Qiuxia Lai, and Qiang Xu. Time series is a special sequence: Forecasting with sample convolution and interaction. *arXiv preprint arXiv:2106.09305*, 2021. 1, 9
- [18] Shizhan Liu, Hang Yu, Cong Liao, Jianguo Li, Weiyao Lin, Alex X Liu, and Schahram Dustdar. Pyraformer: Low-complexity pyramidal attention for long-range time series modeling and forecasting. In *International Conference on Learning Representations*, 2021. 1, 2, 3, 4, 9, 10
- [19] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 10012–10022, 2021. 1
- [20] LIU Minhao, Ailing Zeng, LAI Qiuxia, Ruiyuan Gao, Min Li, Jing Qin, and Qiang Xu. T-wavenet: A tree-structured wavelet neural network for time series signal analysis. In *International Conference on Learning Representations*, 2021. 2
- [21] Gábor Petneházi. Recurrent neural networks for time series forecasting. *arXiv preprint arXiv:1901.00069*, 2019. 9
- [22] David Salinas, Valentin Flunkert, and Jan Gasthaus. Deepar: Probabilistic forecasting with autoregressive recurrent networks. *International Journal of Forecasting*, 2017. 2
- [23] Souhaib Ben Taieb, Rob J Hyndman, et al. *Recursive and direct multi-step forecasting: the best of both worlds*, volume 19. Citeseer, 2012. 2
- [24] Sean J. Taylor and Benjamin Letham. Forecasting at scale. *PeerJ Prepr.*, 2017. 2
- [25] Gerrit JJ van den Burg and Christopher KI Williams. An evaluation of change point detection algorithms. *arXiv preprint arXiv:2003.06222*, 2020. 8
- [26] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017. 1, 2, 9
- [27] Qingsong Wen, Tian Zhou, Chaoli Zhang, Weiqi Chen, Ziqing Ma, Junchi Yan, and Liang Sun. Transformers in time series: A survey. *arXiv preprint arXiv:2202.07125*, 2022. 1, 2, 5
- [28] Jiehui Xu, Jianmin Wang, Mingsheng Long, et al. Autoformer: Decomposition transformers with autocorrelation for long-term series forecasting. *Advances in Neural Information Processing Systems*, 34, 2021. 1, 2, 3, 4, 5, 9
- [29] Ailing Zeng, Xuan Ju, Lei Yang, Ruiyuan Gao, Xizhou Zhu, Bo Dai, and Qiang Xu. Deciwatoh: A simple baseline for 10x efficient 2d and 3d pose estimation. *arXiv preprint arXiv:2203.08713*, 2022. 1
- [30] Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Hui Xiong, and Wancai Zhang. Informer: Beyond efficient transformer for long sequence time-series forecasting. In *The Thirty-Fifth AAAI Conference on Artificial Intelligence, AAAI 2021, Virtual Conference*, volume 35, pages 11106–11115. AAAI Press, 2021. 1, 2, 3, 4, 5, 9
- [31] Tian Zhou, Ziqing Ma, Qingsong Wen, Xue Wang, Liang Sun, and Rong Jin. Fedformer: Frequency enhanced decomposed transformer for long-term series

forecasting. In *International Conference on Machine Learning*, 2022. [1](#), [2](#), [3](#), [4](#), [5](#), [9](#)